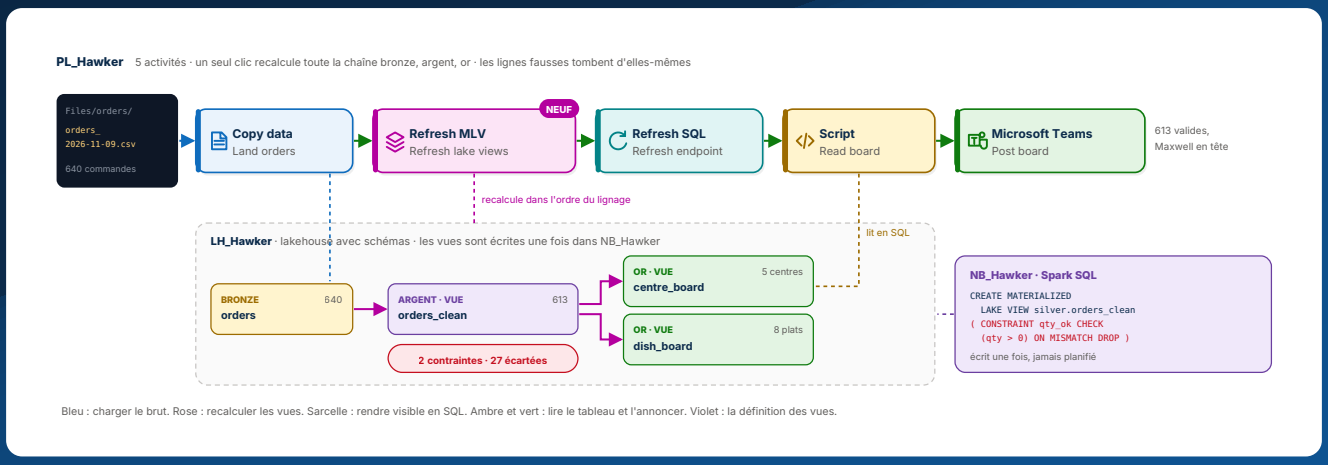
MICROSOFT FABRIC · DATA FACTORY · MATERIALIZED LAKE VIEWS · GUIDE DE RÉFÉRENCE

Pipeline Microsoft Fabric : des vues qui se nettoient seules

CHARGER RECALCULER FILTRER LIRE ANNONCER

Singapour, lundi 9 novembre 2026. Cinq hawker centres, 640 commandes, dont 27 fausses : annulées, remboursées, sans prix. Je vous montre comment écrire trois vues une seule fois, et laisser une activité tout recalculer.



5	3	640	27	0
activités	vues matérialisées	commandes	écartées seules	ligne de nettoyage

AUTEUR
Abdoul Hamid Diallo, Microsoft Data & AI Engineer

FICHIERS DU PROJET
orders_2026-11-09.csv, 01_notebook_mlv.sql, 02_read_board.sql, 03_controle.sql, expressions.txt

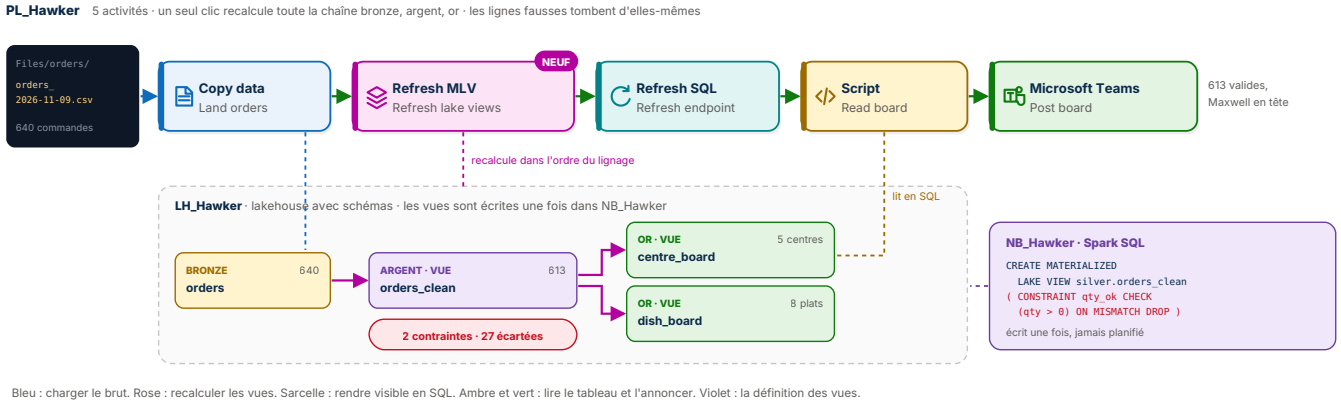
PRÉREQUIS
Un espace de travail Fabric sur une capacité, Teams avec votre compte, aucune notion de Spark

DURÉE
Environ 30 minutes, en quatre étapes vérifiées une à une

VUE D'ENSEMBLE

Ce que vous construisez

Le scénario : chaque soir, les caisses des hawker centres de Singapour envoient la journée dans un CSV, une ligne par commande. Dedans, il y a du vrai et du faux : des assiettes annulées (quantité 0), des remboursements (quantité négative), des prix oubliés. Au lieu d'écrire du nettoyage dans le pipeline, je vous fais déclarer trois **vues matérialisées** (materialized lake views) : une vue argent qui type et **rejette les lignes fausses grâce à deux contraintes**, deux vues or qui font les totaux. Le pipeline charge le brut, puis **une seule activité recalcule toute la chaîne** dans l'ordre du lignage. Ce que vous allez apprendre : **les vues matérialisées du lac**, **les contraintes qui écartent les lignes fausses**, **le lignage calculé par Fabric** et **l'activité Refresh materialized lake view**.



AVANT DE COMMENCER : 30 SECONDES DE VÉRIFICATION

Créez un pipeline vide, ruban **Activités**, cherchez `materialized`. Puis **Nouvel élément**, **Lakehouse** : la case **Schémas de lakehouse** doit être proposée.

ÉLÉMENT	OÙ LE VOIR	SI IL MANQUE
Refresh materialized lake view NEUF	Ruban **Activités**, recherche `materialized`	Plan B page 7 : une activité Notebook qui relance `NB_Hawker`
Schémas de lakehouse	Case à la création du lakehouse	Indispensable : sans schémas, pas de vues matérialisées
Refresh SQL analytics endpoint	Ruban **Activités**, recherche `Refresh SQL`	Une activité Wait de 90 s

Les activités

ACTIVITÉ	TYPE	RÔLE
`Land orders`	Copy data	Le CSV du jour vers la table brute `bronze.orders`, tout en texte
`Refresh lake views`	Refresh materialized lake view	Recalcule `silver.orders_clean`, puis les deux vues or, dans l'ordre du lignage
`Refresh endpoint`	Refresh SQL endpoint	Rend les vues à jour visibles tout de suite en SQL
`Read board`	Script	Sur `WH_Hawker` : brut, valide, meilleur centre, plat roi, total
`Post board`	Microsoft Teams	Annonce le tableau du jour

POURQUOI J'AIME CE MODÈLE D'habitude, chaque calcul est une activité de plus. Ici, la logique vit dans les vues, et le pipeline ne fait qu'un geste : « recalcule ». Fabric déduit seul que l'argent passe avant l'or, en lisant les `FROM`.

DEUX EXÉCUTIONS 1 : charger et recalculer, puis regarder le lignage. 2 : le pipeline complet, jusqu'au message Teams.

ÉTAPE 1

Le lakehouse à schémas, les commandes, l'entrepôt

1 · OBJETS | 2 · VUES | 3 · RECALCULER | 4 · ANNONCER

1 LH_Hawker — LAKEHOUSE

1. Créez un espace de travail **Fabric_Pipeline_15**. **Nouvel élément**, **Lakehouse**, nom `LH_Hawker`, **cochez Schémas de lakehouse**.
2. Dans **Fichiers**, **Nouveau sous-dossier** : `orders`.
3. Dans `orders`, **Charger des fichiers** : `orders_2026-11-09.csv`.

2 WH_Hawker — ENTREPÔT

1. **Nouvel élément**, **Entrepôt**, nom `WH_Hawker`. Il reste vide : il sert à lire le lakehouse en SQL depuis le pipeline.

CENTRE	CE QU'ON Y MANGE
Maxwell	le chicken rice qui fait la queue
Lau Pa Sat	la rue des satay, le soir
Old Airport Road	le char kway teow
Tiong Bahru	les petits-déjeuners et le kaya toast
Chomp Chomp	le hokkien mee de nuit

Données fictives : commandes, prix en dollars de Singapour (SGD) et volumes sont fixés pour l'exercice.

Les premières commandes :

```
order_id,centre,dish,qty,unit_price_sgd,ordered_at
15602,Old Airport Road,Nasi lemak,1,4.50,2026-11-09 07:01:00
15516,Lau Pa Sat,Nasi lemak,3,4.50,2026-11-09 07:02:00
15076,Maxwell,Chicken rice,1,5.00,2026-11-09 07:05:00
15454,Maxwell,Satay,30,1.20,2026-11-09 07:05:00
15023,Maxwell,Hokkien mee,3,6.00,2026-11-09 07:06:00
15141,Old Airport Road,Chicken rice,1,5.00,2026-11-09 07:12:00
… 640 commandes
```

LIGNE FAUSSE	COMBIEN	CONTRAINTE QUI L'ÉCARTE
Assiette annulée, `qty` = 0	9	`qty_ok`
Remboursement, `qty` négative	9	`qty_ok`
Prix vide	9	`price_ok`
Total	**27**	sur 640 lignes

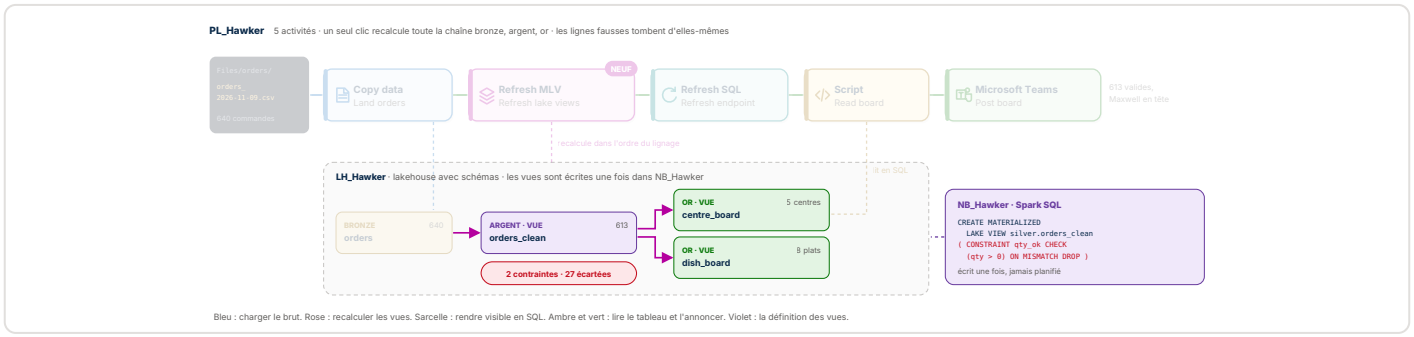
CE QUE LE GÉRANT VEUT VOIR Le meilleur centre, le plat qui rapporte le plus et le total du jour, calculés sur les seules commandes valides.

AVANT LES VUES ☐ case Schémas cochée ☐ le CSV dans `orders` ☐ `WH_Hawker` créé ☐ même espace

ÉTAPE 2

Écrire les vues une seule fois

1 · OBJETS | 2 · VUES | 3 · RECALCULER | 4 · ANNONCER



3 NB_Hawker

NOTEBOOK · SPARK SQL

C'est la seule fois où l'on écrit du Spark SQL, et vous n'avez qu'à coller. Dans `LH_Hawker` : **Ouvrir le notebook**, **Nouveau notebook**, nom `NB_Hawker`. Langage du notebook : **Spark SQL**. Quatre cellules, lignes **[Cellule 1]** à **[Cellule 4]** de `01_notebook_mlv.sql`, puis **Exécuter tout**. Quand les quatre cellules ont leur coche verte, onglet **Exécuter**, **Arrêter la session** : la capacité est libérée pour le pipeline.

Cellule 1 : les couches et la table brute

```
CREATE SCHEMA IF NOT EXISTS bronze;
CREATE SCHEMA IF NOT EXISTS silver;
CREATE SCHEMA IF NOT EXISTS gold;
CREATE TABLE IF NOT EXISTS bronze.orders (order_id STRING,
centre STRING, dish STRING, qty STRING, unit_price_sgd STRING,
ordered_at STRING);
```

Cellules 3 et 4 : l'or

```
CREATE OR REPLACE MATERIALIZED LAKE VIEW gold.centre_board
AS SELECT centre, COUNT(*) AS orders, SUM(qty) AS portions,
CAST(SUM(amount) AS DECIMAL(10,2)) AS revenue_sgd
FROM silver.orders_clean
GROUP BY centre;

CREATE OR REPLACE MATERIALIZED LAKE VIEW gold.dish_board
AS SELECT dish, SUM(qty) AS portions, CAST(SUM(amount) AS
DECIMAL(10,2)) AS revenue_sgd
FROM silver.orders_clean
GROUP BY dish;
```

Cellule 2 : l'argent, avec ses deux contraintes

```
CREATE OR REPLACE MATERIALIZED LAKE VIEW silver.orders_clean
(
  CONSTRAINT qty_ok CHECK (qty > 0) ON MISMATCH DROP,
  CONSTRAINT price_ok CHECK (unit_price IS NOT NULL) ON MISMATCH DROP
)
COMMENT "Commandes valides des hawker centres"
AS SELECT
  CAST(order_id AS INT) AS order_id,
  centre,
  dish,
  CAST(qty AS INT) AS qty,
  CAST(unit_price_sgd AS DECIMAL(6,2)) AS unit_price,
  CAST(ordered_at AS TIMESTAMP) AS ordered_at,
  CAST(qty AS INT) * CAST(unit_price_sgd AS DECIMAL(6,2)) AS amount
FROM bronze.orders;
```

LA LIGNE QUI REMPLACE LE NETTOYAGE `CHECK (qty > 0) ON MISMATCH DROP` : toute ligne qui ne passe pas est écartée au recalcul, sans erreur. Avec `FAIL` à la place de `DROP`, le recalcul s'arrêterait au contraire.

POURQUOI BRONZE EST EN TEXTE `Land orders` copie le CSV tel quel. Le typage (`CAST`) se fait dans la vue argent : un prix vide devient `NULL`, et `price_ok` l'écarte.

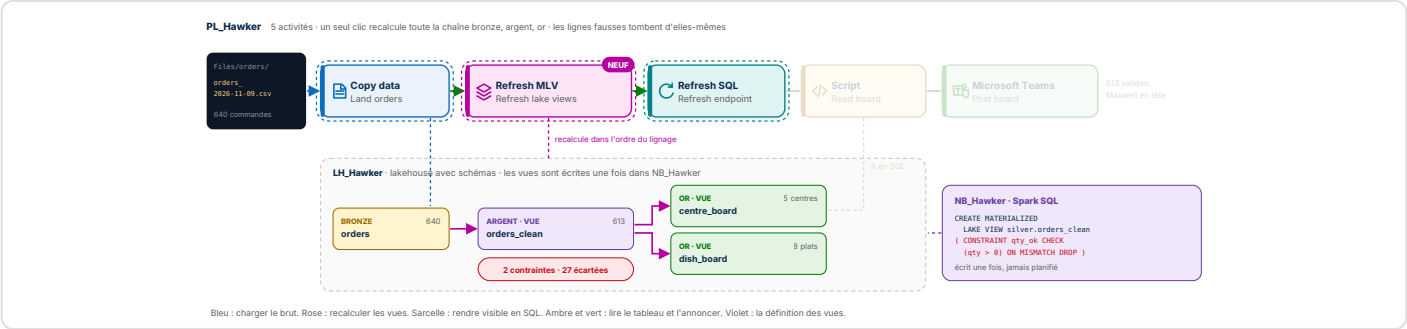
Vérification le lignage

Dans `LH_Hawker`, ruban : **Vues matérialisées du lac** (Materialized lake views), **Gérer** (Manage). Le lignage s'affiche : `orders` vers `orders_clean`, puis vers `centre_board` et `dish_board`. Les vues sont vides : bronze l'est encore. C'est normal.

ÉTAPE 3

Charger, recalculer, rendre visible

1 · OBJETS | 2 · VUES | **3 · RECALCULER** | 4 · ANNONCER



4 Land orders

COPY DATA

Nouvel élément, **Pipeline**, `PL_Hawker`. **Copier les données**, nom `Land orders`.

Source	`LH_Hawker`, **Fichiers**, dossier `orders`, fichier `orders_2026-11-09.csv`
Format	**Texte délimité**, en-tête coché
Destination	`LH_Hawker`, **Tables**, schéma `bronze`, table existante `orders`
Action de table	**Remplacer**

5 Refresh lake views

REFRESH MLV · NEUF

1. Ruban **Activités**, cherchez `materialized` : **Refresh materialized lake view**. Nom `Refresh lake views`, relié à `Land orders` par la flèche verte.
2. Onglet **Paramètres**, **Connexion** : **Parcourir tout**, **Nouveau**, **Lakehouse**, compte d'organisation, nom `CN_Lakehouse`. Une seule connexion sert pour tous vos lakehouses.
3. **Espace de travail** : `Fabric_Pipeline_15`. **Lakehouse** : `LH_Hawker`.

6 Refresh endpoint

REFRESH SQL

Relié à `Refresh lake views`. Même connexion `CN_Lakehouse`, espace `Fabric_Pipeline_15`, point de terminaison `LH_Hawker`.

CE QUE FAIT L'ACTIVITÉ NEUVE Elle recalcule **toutes** les vues du lakehouse, sans que vous les nommiez. L'ordre vient du lignage : `orders_clean` d'abord, puis les deux vues or. Ajoutez une quatrième vue demain : rien à changer dans le pipeline.

Exécution 1 — charger et recalculer

Enregistrer, **Exécuter**. Trois activités vertes. Puis `LH_Hawker`, **Vues matérialisées du lac**, **Gérer** : le lignage est vert, l'onglet des exécutions récentes montre le recalcul.

Le chiffre à voir — point de terminaison SQL

Ouvrez le point de terminaison SQL de `LH_Hawker`, requête **[Contrôle]** :

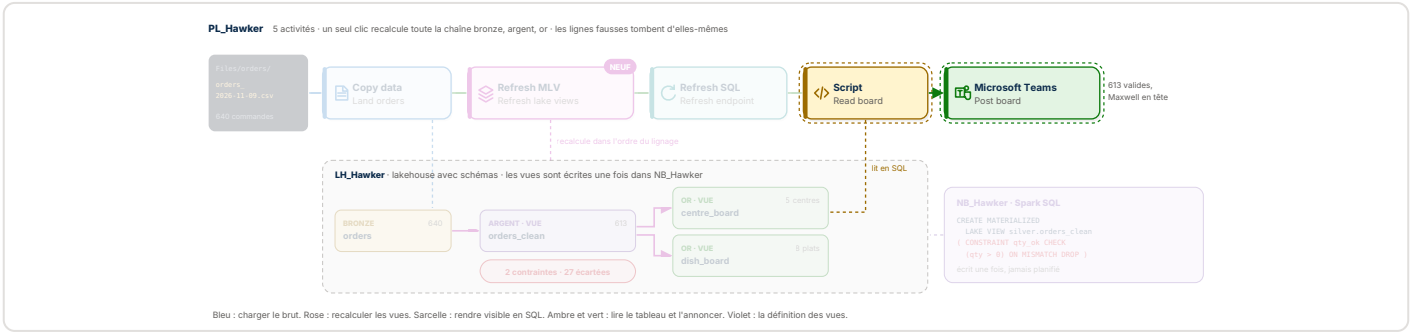
```
Maxwell 150 · 565 · 1755.70
Old Airport Road 122 · 468 · 1412.60
Tiong Bahru 118 · 470 · 1359.80
Lau Pa Sat 114 · 426 · 1252.60
Chomp Chomp 109 · 310 · 1157.30
```

613, PAS 640 Les cinq centres totalisent 613 commandes. Les 27 lignes fausses ne sont plus là, et vous n'avez écrit aucun `WHERE`.

ÉTAPE 4

Lire le tableau et l'annoncer

1 · OBJETS | 2 · VUES | 3 · RECALCULER | 4 · ANNONCER



7 Read board — SCRIPT

Relié à `Refresh endpoint`. Connexion `WH_Hawker`, type **Requête**, le contenu de `02_read_board.sql` (ligne **[Read board]**). La dernière colonne, `msg`, contient déjà la phrase complète pour Teams.

```
WITH b AS (
  SELECT
    (SELECT COUNT(*) FROM LH_Hawker.bronze.orders) AS raw_rows,
    (SELECT COUNT(*) FROM LH_Hawker.silver.orders_clean) AS clean_rows,
    (SELECT TOP 1 centre FROM LH_Hawker.gold.centre_board
       ORDER BY revenue_sgd DESC) AS top_centre,
    (SELECT CAST(MAX(revenue_sgd) AS varchar(12))
       FROM LH_Hawker.gold.centre_board) AS top_centre_sgd,
    (SELECT TOP 1 dish FROM LH_Hawker.gold.dish_board
       ORDER BY revenue_sgd DESC) AS top_dish,
    (SELECT CAST(MAX(revenue_sgd) AS varchar(12))
       FROM LH_Hawker.gold.dish_board) AS top_dish_sgd,
    (SELECT CAST(SUM(revenue_sgd) AS varchar(12))
       FROM LH_Hawker.gold.centre_board) AS total_sgd
)
SELECT *,
  CONCAT('Hawker board, 9 novembre 2026 : ',
    clean_rows, ' commandes valides sur ', raw_rows,
    ' (', raw_rows - clean_rows, ' écartées par les contraintes). ',
    'Meilleur centre : ', top_centre, ', ', top_centre_sgd, ' SGD. ',
    'Plat roi : ', top_dish, ', ', top_dish_sgd, ' SGD. ',
    'Total : ', total_sgd, ' SGD.') AS msg
FROM b;
```

TROIS PARTIES DANS LE NOM
`LH_Hawker.silver.orders_clean` : lakehouse, schéma, vue. Depuis l'entrepôt, les vues se lisent comme des tables. Les montants passent en texte pour garder leurs deux décimales.

8 Post board — TEAMS

1. **Microsoft Teams**, nom `Post board`, relié à `Read board`.
2. Votre connexion Teams, publier dans une conversation ou un canal, titre `Hawker board`.
3. **Message** : collez directement la ligne **[Teams message]** de `expressions.txt` : `@{activity('Read board').output.resultSets[0].rows[0].msg}`

Exécution 2 — le pipeline complet

Cinq activités vertes. Sortie de `Read board` :

```
"raw_rows": 640, "clean_rows": 613,
"top_centre": "Maxwell", "top_centre_sgd":
"1755.70", "top_dish": "Satay", "top_dish_sgd":
"1500.00", "total_sgd": "6938.00"
```

Le message arrive dans Teams, capture ci-dessous.

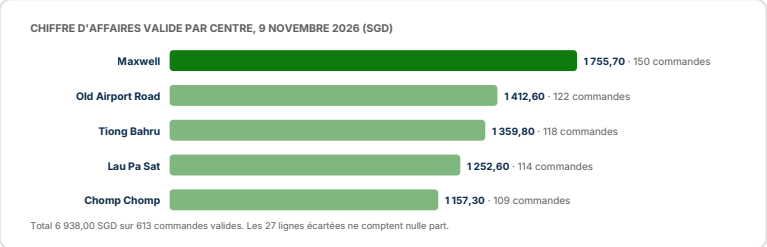
LE SATAY EN TÊTE Le chicken rice est le plat le plus commandé, mais les brochettes de satay partent par 10, 20 ou 30 : 1 250 brochettes, 1 500 SGD.

Le résultat — dans Teams



RÉFÉRENCE

Vérifier, plan B, débloquer



640 → 613
commandes brutes, puis valides, sans une ligne de nettoyage

27
lignes écartées par `qty_ok` (18) et `price_ok` (9)

6 866,60
SGD si l'on somme le brut : remboursements déduits, prix vides ignorés

6 938,00
SGD sur les seules commandes valides

Ce qu'attend le contrôle

Requête **[Contrôle]** sur le point de terminaison SQL de `LH_Hawker`.

CENTRE	CMD	PORTIONS	SGD
Maxwell	150	565	1 755,70
Old Airport Road	122	468	1 412,60
Tiong Bahru	118	470	1 359,80
Lau Pa Sat	114	426	1 252,60
Chomp Chomp	109	310	1 157,30
Total	**613**	**2 239**	**6 938,00**

PLAN B · PAS D'ACTIVITÉ REFRESH MLV Activité **Notebook** à la place de `Refresh lake views`, notebook `NB_Hawker`. Ses `CREATE OR REPLACE` reconstruisent les trois vues sur le brut du jour : même résultat, un peu plus lent.

Si quelque chose bloque

SYMPTÔME	CAUSE ET CORRECTION
Cellule 2 : « MATERIALIZED LAKE VIEW » non reconnu	Lakehouse créé sans schémas, ou runtime trop ancien. **Paramètres de l'espace**, **Data Engineering**, runtime **1.3**. Sinon, recréez `LH_Hawker` avec la case cochée.
Land orders : le schéma `bronze` n'apparaît pas	Lancez d'abord `NB_Hawker` : la cellule 1 crée `bronze.orders`. Puis actualisez la liste des tables.
Refresh lake views : « artifactDmtsCredential »	Choisissez la connexion Lakehouse (`CN_Lakehouse`), pas l'élément `LH_Hawker`.
Refresh lake views : `MLV_SPARK_JOB_CAPACITY_THROTTLING`	La capacité n'a plus de place Spark. Arrêtez la session du notebook (onglet **Exécuter**), annulez ce qui tourne dans **Surveiller**, attendez deux minutes, relancez.
Teams affiche l'expression au lieu du message	Il manque les accolades : `@{…}`, pas `@activity(…)`.
Contrôle : 640 commandes au lieu de 613	Cellule 2 collée sans le bloc `CONSTRAINT`. Recollez-la entière, relancez la cellule, puis le pipeline.